

Core Java Interview Notes (Short Answers)

1. What is Java?

Java is a high-level, object-oriented programming language developed by Sun Microsystems. It follows the principle "Write Once, Run Anywhere" (WORA).

Features of Java

- Platform Independent
 - Object-Oriented
 - Secure
 - Robust
 - Multithreaded
 - Portable
 - High Performance
-

2. JVM, JRE, JDK

JVM (Java Virtual Machine)

Executes Java bytecode and makes Java platform independent.

JRE (Java Runtime Environment)

Contains JVM + libraries required to run Java applications.

JDK (Java Development Kit)

Contains JRE + development tools like javac, java, javadoc.

Interview Question

JDK = JRE + Development Tools

JRE = JVM + Libraries

JVM = Executes Bytecode

3. Data Types

Primitive Data Types

Type	Size
byte	1 byte
short	2 bytes
int	4 bytes
long	8 bytes
float	4 bytes
double	8 bytes
char	2 bytes
boolean	JVM dependent

Non-Primitive Types

- String
 - Array
 - Class
 - Interface
-

4. Variables

Local Variable

Declared inside method.

Instance Variable

Declared inside class but outside method.

Static Variable

Shared among all objects.

5. Type Casting

Implicit Casting

Smaller type → Larger type.

Example: int → long

Explicit Casting

Larger type → Smaller type.

Example: double → int

6. OOP Concepts

Class

Blueprint of an object.

Object

Instance of a class.

Example: Car c = new Car();

Encapsulation

Wrapping data and methods into a single unit.

Benefit: Data Security

Inheritance

Acquiring properties of parent class.

Keyword: extends

Benefit: Code Reusability

Polymorphism

Compile Time

Method Overloading

Run Time

Method Overriding

Abstraction

Showing essential details and hiding implementation.

Achieved by:

- Abstract Class
 - Interface
-

7. Constructor

Special method used to initialize objects.

Characteristics:

- Same name as class
- No return type

Types:

- Default Constructor
 - Parameterized Constructor
-

8. this Keyword

Refers to current object.

Uses:

- Access instance variables
 - Call current constructor
-

9. super Keyword

Refers to parent class object.

Uses:

- Access parent variables
 - Call parent constructor
-

10. static Keyword

Belongs to class rather than object.

Examples:

- Static Variable
 - Static Method
 - Static Block
-

11. final Keyword

Used to restrict modification.

final Variable

Cannot change value.

final Method

Cannot override.

final Class

Cannot inherit.

Example: String class is final.

12. Arrays

Collection of similar data types stored in contiguous memory.

Example: `int[] arr = {1,2,3};`

Advantages:

- Fast access
 - Fixed size
-

13. String

String is a sequence of characters.

Example: `String name = "Java";`

Important Point

String is Immutable.

Why String is Immutable?

Security, Thread Safety, Performance.

String Pool

Special memory area inside Heap where string literals are stored.

Example: `String s1="Java"; String s2="Java";`

Both refer to same object.

14. String vs StringBuilder vs StringBuffer

String

Immutable

StringBuilder

Mutable Not Thread Safe Fast

StringBuffer

Mutable Thread Safe Slower than StringBuilder

15. Wrapper Classes

Convert primitive types into objects.

Examples:

- Integer
- Double
- Float
- Character

Autoboxing

Primitive → Object

Unboxing

Object → Primitive

16. Exception Handling

Exception is an abnormal condition during program execution.

Exception Hierarchy

Throwable | |-- Error | |-- Exception

Checked Exception

Checked at compile time.

Examples:

- IOException
 - SQLException
-

Unchecked Exception

Occurs at runtime.

Examples:

- NullPointerException

- ArithmeticException
-

Keywords

try

Contains risky code.

catch

Handles exception.

finally

Always executes.

throw

Throws exception manually.

throws

Declares exception.

17. Collections Framework

Framework used to store and manipulate groups of objects.

List

ArrayList

- Dynamic Array
- Fast Retrieval

LinkedList

- Doubly Linked List
- Fast Insertion/Deletion

Vector

- Synchronized
-

Set

HashSet

- No duplicates
- Unordered

LinkedHashSet

- Maintains insertion order

TreeSet

- Sorted order
-

Map

HashMap

- Key-Value pair
- One null key allowed

LinkedHashMap

- Maintains insertion order

TreeMap

- Sorted by keys

Hashtable

- Synchronized
 - No null key
-

18. HashMap Internal Working

1. Key's hashCode() generated.
2. Hash converted into bucket index.
3. Data stored in bucket.
4. Collision handled using Linked List / Tree.

Most Asked Interview Question.

19. equals() vs ==

==

Compares memory references.

equals()

Compares object contents.

20. hashCode()

Returns integer hash value of object.

Rule: Equal objects must have same hashCode().

21. Comparable vs Comparator

Comparable

Used for default sorting.

Method: compareTo()

Comparator

Used for custom sorting.

Method: compare()

22. Multithreading

Executing multiple tasks simultaneously.

Benefits:

- Better Performance
 - Efficient CPU Usage
-

Thread Creation

Extending Thread Class

Implementing Runnable Interface

Preferred: Runnable Interface

23. Thread Lifecycle

New ↓ Runnable ↓ Running ↓ Waiting/Blocked ↓ Terminated

24. Synchronization

Used to avoid data inconsistency in multithreading.

Keyword: synchronized

25. Deadlock

Situation where two threads wait for each other forever.

26. Java 8 Features

Lambda Expression

Used to write concise code.

Functional Interface

Contains only one abstract method.

Examples:

- Runnable
- Comparator

Annotation: @FunctionalInterface

Stream API

Used for processing collections.

Methods:

- filter()
 - map()
 - sorted()
 - collect()
 - reduce()
-

Optional Class

Used to avoid NullPointerException.

27. File Handling

Classes:

- File
- FileReader
- FileWriter
- BufferedReader
- BufferedWriter

Used for reading and writing files.

28. Serialization

Converting object into byte stream.

Interface: Serializable

Used for:

- Saving object state
 - Network transfer
-

29. Garbage Collection

Automatic memory cleanup process.

Method: System.gc()

Removes unused objects from heap memory.

30. Memory Areas

Heap Memory

Stores Objects.

Stack Memory

Stores Method Calls and Local Variables.

Interview Question

Objects → Heap

Local Variables → Stack

31. Access Modifiers

Modifier	Same Class	Same Package	Subclass	Other Package
private	Yes	No	No	No
default	Yes	Yes	No	No
protected	Yes	Yes	Yes	No
public	Yes	Yes	Yes	Yes

32. Abstract Class vs Interface

Abstract Class:

- Can have abstract and concrete methods
- Supports constructor

Interface:

- Supports multiple inheritance
- All methods abstract by default (before Java 8)

Interview Favorite Question

33. Important Java Interview Questions

1. Difference between JDK, JRE, JVM?
2. Why Java is Platform Independent?
3. What is OOP?
4. Explain Encapsulation.
5. Difference between Overloading and Overriding?
6. Why String is Immutable?
7. String vs StringBuilder vs StringBuffer?
8. ArrayList vs LinkedList?
9. HashMap Internal Working?
10. HashMap vs Hashtable?
11. equals() vs == ?
12. Comparable vs Comparator?
13. Checked vs Unchecked Exception?
14. Runnable vs Thread?
15. Synchronization in Java?
16. What is Deadlock?
17. What is Lambda Expression?
18. What is Stream API?
19. Heap vs Stack?
20. Abstract Class vs Interface?